



VIZNET

VizChoice

-

NetRadio

-

PAL SDK Integration

Technical documentation

V 1

Document version	2
Integration note for FAST channel publishers	3
1. Objective	3
2. Context	3
3. Functional principle	4
4. When this implementation is required	4
5. Player-side requirements	5
5.1 Create a PAL nonce	5
5.2 Retrieve the nonce	5
5.3 Append givn to the CDN Edge request	5
6. CDN Edge / Broadpeak forwarding requirements	5
6.1 Preserve givn exactly as received	6
6.2 Forward givn to Broadpeak and downstream ad systems	6
6.3 Support multi-hop propagation	6
6.4 Use intermediary macros if needed	6
7. Example request pattern	6
7.1 Player request to CDN Edge	6
7.2 Downstream Google-facing request	7
8. Playback event signaling	7
9. Implementation guidance for the publisher	8
Mandatory	8
Recommended	8
10. Documentation references	8

Document version

Version	Date	Modification	Author
V1	2026.03.18	Création.	Pierre Raffalli

Integration note for FAST channel publishers

1. Objective

This document explains how a FAST channel publisher should implement the Google PAL SDK in a player environment where ad requests are routed through the Broadpeak CDN Edge before reaching SSAI / ad decisioning components.

The goal is to:

- generate a PAL nonce on the client side,
- **append that nonce to the outbound ad-related request using the “givn” query parameter,**
- **ensure the CDN Edge forwards givn unchanged to Broadpeak and any downstream ad servers that may call Google demand,**
- preserve Google programmatic eligibility when Google demand is reachable through the monetization chain.

<https://developers.google.com/ad-manager/pal/html5/get-started>

2. Context

In this setup, the player does **not** rely on the Google IMA SDK for ad playback. Instead, the publisher uses:

- a custom player or SSAI-compatible playback stack,
- the **Google Programmatic Access Library (PAL SDK)** to generate a secure nonce,
- Broadpeak CDN Edge as the request entry point,
<https://developers.google.com/ad-manager/pal>
- Broadpeak SSAI and/or downstream ad systems (like VizChoice) for ad decisioning and ad stitching.

Google states that PAL is intended for environments where publishers want access to Google programmatic demand **without implementing IMA**.

3. Functional principle

The PAL SDK generates an encrypted **nonce** for a playback session.
That nonce must be appended to the Google-facing ad request using:

```
givn=<PAL_NONCE>
```

Google recommends using **givn** parameter.

In a Broadpeak / CDN Edge setup, the expected behavior is:

Player

- > PAL SDK generates nonce
- > Player calls CDN Edge **with** givn=<nonce>
- > CDN Edge forwards givn to Broadpeak SSAI / ad proxy
- > Downstream system forwards givn to any Google-facing ad request

If there are multiple intermediaries, Google requires the encrypted nonce to be forwarded through **each intermediary** until it reaches the final Google ad request.

4. When this implementation is required

This implementation is relevant only if the monetization chain can ultimately reach Google demand such as Google Ad Manager / Authorized Buyers / DV360 demand through an intermediary.

It is useful in cases such as:

- Player → CDN Edge → Broadpeak → ad proxy / SSP → Google demand
- Player → CDN Edge → Broadpeak → third-party ad server → Google Ad Manager

It is not useful if the request chain never reaches any Google ad-serving or Google demand endpoint. In that case, the **givn** parameter will simply be ignored.

5. Player-side requirements

The publisher player must integrate the PAL SDK on the relevant platform and perform the following steps:

5.1 Create a PAL nonce

The player initializes PAL and builds a **NonceRequest** with the required playback and environment metadata.

Typical fields depend on platform, but generally include:

- player name / version,
- video dimensions,
- autoplay / mute state where relevant,
- partner / OMID-related information if supported,
- privacy / consent settings where applicable.

5.2 Retrieve the nonce

The player uses the **PAL NonceLoader / NonceManager** flow and retrieves the nonce via `getNonce()` method.

5.3 Append givn to the CDN Edge request

The player appends the generated nonce as a query parameter on the CDN Edge request:

```
https://cdn-edge.example.com/index.m3u8?...&givn=<PAL_NONCE>
```

This is the key integration requirement.

6. CDN Edge / Broadpeak forwarding requirements

The CDN Edge receives player query parameters and automatically relays them to Broadpeak SSAI and to the VizChoice ad proxy.

For PAL support, the CDN Edge and all downstream components must follow these rules:

6.1 Preserve **givn** exactly as received

The **givn** parameter must be forwarded **unchanged**.

It must not be renamed, truncated, decoded, regenerated, or filtered out.

6.2 Forward **givn** to Broadpeak and downstream ad systems

Any request path that may eventually result in a Google-facing ad call must preserve **givn**.

6.3 Support multi-hop propagation

If Broadpeak calls another ad server, SSP, or ad proxy before Google demand is reached, the intermediary must also pass through the PAL nonce.

Google explicitly documents this multi-hop requirement.

6.4 Use intermediary macros if needed

If a downstream third-party ad server requires macro substitution, it should map the incoming player nonce to the final Google request. Google gives the example of a custom macro replaced into **givn=** on the Google-facing request

7. Example request pattern

7.1 Player request to CDN Edge

```
https://cdn-url/index.m3u8?
bpkio_serviceid=<UUID>&
bpkio_sessionid=<UUID>&
channel_code=<CHANNEL_CODE>&
cid=<CONTENT_ID>&
stream_id=<STREAM_ID>&
session_id=<SESSION_ID>&
gdpr=<0_or_1>&
gdpr_consent=<TCF_STRING>&
```

```
addtl_consent=<AC_STRING>&  
lmt=<0_or_1>&  
context=linear&  
ifa=<DEVICE_ID>&  
ifa_type=<IFA_TYPE>&  
givn=<PAL_NONCE>
```

7.2 Downstream Google-facing request

If a downstream system eventually calls Google, the final request must still contain:

```
...&givn=<PAL_NONCE>
```

8. Playback event signaling

Implementing **givn** propagation is only part of the PAL integration. Google also requires the player to notify PAL about playback lifecycle and ad interactions.

At minimum, the player should wire the following events:

- `sendPlaybackStart()`
- `sendPlaybackEnd()`
- `sendAdClick()`
- `sendAdTouch()` where applicable.

These events should be connected to the real player lifecycle, not only to ad-request creation.

<https://developers.google.com/ad-manager/pal/html5/get-started>

9. Implementation guidance for the publisher

The publisher should implement the following:

Mandatory

- Integrate the PAL SDK in the playback application.
- Generate one PAL nonce per playback session / stream request according to Google guidance.
- Append givn to the CDN Edge request.
- Ensure givn is forwarded through CDN Edge, Broadpeak, and any downstream intermediary that may reach Google demand.
- Wire PAL playback event callbacks in the player.

Recommended

- Log presence / absence of givn at each hop for debugging.
- Validate that the final Google-facing request contains givn.
- Validate behavior separately by platform: web, Android TV, tvOS, Cast, etc., depending on the publisher footprint.

10. Documentation references

Google PAL SDK overview:

Google documents PAL as the lightweight library for requesting programmatic ads without relying on IMA.

<https://developers.google.com/ad-manager/pal>

Google PAL SDK HTML5 get started:

This guide documents nonce generation, givn usage, playback event signaling, and forwarding through third-party ad servers.

<https://developers.google.com/ad-manager/pal/html5/get-started>

Google PAL SDK Android get started:

Useful reference for Android / Android TV implementations and nonce handling.

<https://developers.google.com/ad-manager/pal/android/get-started>

